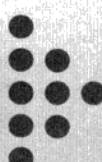




第15章

反 射

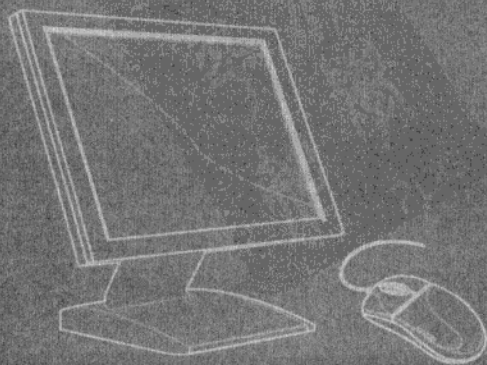


学前提示

所谓反射，可以理解为在运行时获取对象类型信息的操作。传统的编程方法要求程序员在编译阶段就决定使用的类型，但是在反射机制的帮助下，编程人员可以在运行时动态地获取这些信息，从而编写更加具有可移植性的代码。了解反射机制对学习 Java 高级应用大有益处。

知识要点

- 什么是反射
- Java 反射 API
- 使用反射获取类信息
- 使用反射创建对象
- 使用反射调用方法和操作成员变量
- 代理模式



15.1 反射概述

反射(reflection)的概念是由 Smith 在 1982 年首次提出的,主要是指程序可以访问、检测和修改它本身状态或行为的一种能力。这一概念的提出很快引发了计算机科学领域关于应用反射性的研究。它首先被程序语言的设计领域所采用。

在计算机科学领域,反射是指一类应用,它们能够自描述和自控制。也就是说,这类应用通过采用某种机制来实现对自己行为的描述(self-representation)和监测(examination),并能根据自身行为的状态和结果,调整或修改应用所描述行为的状态和相关的语义。

15.1.1 Java 中的反射机制

反射是 Java 程序开发语言的特征之一。它允许动态地发现和绑定类、方法、字段,以及所有其他的由语言所产生的元素。反射可以做的不仅仅是简单地列举类、字段以及方法。通过反射,还能够在需要时完成创建实例、调用方法以及访问字段的工作。反射是 Java 被视为动态(或准动态)语言的关键。

归纳起来,Java 反射机制主要提供了以下功能。

- 在运行时判断任意一个对象所属的类。
- 在运行时构造任意一个类的对象。
- 在运行时判断任意一个类所具有的成员变量和方法。
- 在运行时调用任意一个对象的方法。通过反射甚至可以调用到 private 的方法。
- 生成动态代理。

15.1.2 Java 反射 API

Java 反射所需要的类并不多,主要有 java.lang.Class 类和 java.lang.reflect 包中的 Field、Constructor、Method、Array 类,下面对这些类做一个简单的说明。

- Class 类: Class 类的实例是表示正在运行的 Java 应用程序中的类和接口。
- Field 类: 提供有关类或接口的属性的信息,以及对它的动态访问权限。反射的字段可能是一个类(静态)属性或实例属性,简单的理解可以把它看成一个封装反射类的属性的类。
- Constructor 类: 提供关于类的单个构造方法的信息以及对它的访问权限。这个类和 Field 类不同,Field 类封装了反射类的属性,而 Constructor 类则封装了反射类的构造方法。
- Method 类: 提供关于类或接口上单独某个方法的信息。所反映的方法可能是类方法或实例方法(包括抽象方法)。这个类不难理解,它是用来封装反射类方法的一个类。
- Array 类: 提供了动态创建数组和访问数组的静态方法。该类中的所有方法都是静态方法。

其中,Class 类是 Java 反射的起源,针对任何一个你想探勘的类,只有先为它产生一个



Class 类的对象,接下来才能通过 Class 对象获取其他想要的信息。接下来就重点介绍一下 Class 类。

15.1.3 Class 类

Java 程序在运行时,系统会对所有的对象进行所谓的运行时类型标识,用来保存这些类型信息的类就是 Class 类。Class 类封装一个对象和接口运行时的状态。

JVM 为每种类型管理着一个独一无二的 Class 对象。也就是说,每个类(型)都有一个 Class 对象。Java 程序运行过程中,当需要创建某个类的实例时,JVM 首先检查所要加载的类对应的 Class 对象是否已经存在。如果还不存在,JVM 就会根据类名查找对应的字节码文件并加载,接着创建对应的 Class 对象,最后才创建出这个类的实例。

提示

Java 基本数据类型(boolean、byte、char、short、int、long、float 和 double)和关键字 void 也都对应一个 Class 对象。每个数组属性也被映射为 Class 对象,所有具有相同类型和维数的数组都共享该 Class 对象。

也就是说,运行中的类或接口在 JVM 中都会有一个对应的 Class 对象存在,它保存了对应类和接口的类型信息。要想获取类和接口的相应信息,需要先获取这个 Class 对象。

1. 获得 Class 类

有以下 3 种方式可以获取 Class 对象。

(1) 调用 Object 类的 getClass()方法来得到 Class 对象,这也是最常见的产生 Class 对象的方法。如:

```
MyObject x;  
Class c1 = x.getClass();
```

(2) 使用 Class 类的 forName()静态方法获得与字符串对应的 Class 对象。如:

```
Class c2=Class.forName("java.lang.String");
```

注意

参数字符串必须是类或接口的全限定名。

(3) 使用类型名.class 获取该类型对应的 Class 对象。如:

```
Class c11 = Manager.class;  
Class c12 = int.class;  
Class c13 = double[].class;
```

2. 常用方法

Class 类中提供了大量的方法,用来获取所代表的实体(类、接口、数组、枚举、注解、基本类型或 void)的信息。

常用方法如下。



- `public String getName()`: 获取此 `Class` 对象所表示的实体的全限定名。
- `public Field[] getFields()`: 获取此 `Class` 对象所表示的实体的所有 `public` 字段。
- `public Field[] getDeclaredFields()`: 获取此 `Class` 对象所表示的实体的所有字段。
- `public Method[] getMethods()`: 获取此 `Class` 对象所表示的实体的所有 `public` 方法。
- `public Method[] getDeclaredMethods()`: 获取此 `Class` 对象所表示的实体的所有方法。
- `public Method getMethod(String name, Class<?>... parameterTypes)`: 获取此 `Class` 对象特定的方法。`name` 参数指定方法的名字, `parameterTypes` 为可变参数指定方法的参数数据类型。
- `public Constructor<?>[] getConstructors()`: 获取此 `Class` 对象所表示的实体的所有 `public` 构造方法。
- `public Constructor<?>[] getDeclaredConstructors()`: 获取此 `Class` 对象所有的构造方法。
- `public Constructor<T> getDeclaredConstructor(Class<?>... parameterTypes)`: 获取此 `Class` 对象有特定的构造方法。
- `public Class getSuperClass()`: 获取此 `Class` 对象所表示的实体的父类 `Class`。
- `public Class[] getInterfaces()`: 获取此 `Class` 对象所表示的实体实现的所有接口 `Class` 列表。
- `public Annotation[] getAnnotations()`: 获取此元素上存在的所有注释。
- `public Annotation[] getDeclaredAnnotations()`: 获取此元素上直接存在的所有注释。
- `public T newInstance()`: 创建此 `Class` 对象所表示的类的一个新实例。使用的是不带参数的构造方法。

15.2 使用 Java 反射机制

了解了 Java 反射的详细细节之后, 就可以使用反射机制来获取类的详细信息、创建类的对象、访问属性值、调用类的方法等。

15.2.1 获取类型信息

要在运行时获取某个类型的信息, 需要先获取这个类型所对应的 `Class` 对象, 然后调用 `Class` 类提供的相应方法来获取。下面以一个示例来逐步介绍如何利用反射来获取指定类的详细信息。

1. 获取指定类对应的 `Class` 对象

这一步骤很简单, 直接使用 `Class` 类的 `forName()` 静态方法来获取。

```
Class clazz = Class.forName("java.util.ArrayList");
```

2. 获取类的包名

通过调用 `Class` 对象的 `getPackage()` 方法, 可以得到此 `Class` 对象所对应的实体所在的包



的信息描述类 `java.lang.Package` 的一个对象。

`Package` 类包含有关 Java 包的实现和规范的版本信息,通过 `Package` 提供的方法可以访问相关的信息。如下代码是获取该类所在包的全名。

```
String packageName = clazz.getPackage().getName();
```

3. 获取类的修饰符

通过 `Class` 对象的 `getModifiers()`方法可以获得此 `Class` 对象所对应的实体的用整数表示的类修饰符值。

```
int mod = clazz.getModifiers();
```

要想把这个整数值转换成对应的字符串,可以使用 `java.lang.reflect.Modifier` 类提供的 `toString(int mod)`静态方法。

```
String modifier = Modifier.toString(mod);
```

4. 获取类的全限定名

通过 `Class` 对象的 `getName()`方法可以获得此 `Class` 对象所对应的实体的全限定名。

```
String className = clazz.getName();
```

注意,由于历史原因,数组类型的 `getName()`方法会返回奇怪的名字。

5. 获取类的父类

通过 `Class` 对象的 `getSuperClass()`方法可以获得此 `Class` 对象所对应的实体的直接父类 `Class` 对象。

```
Class superClass = clazz.getSuperclass();
```

6. 获取类实现的接口

通过 `Class` 对象的 `getInterfaces()`方法可以获取此 `Class` 对象所对应的实体所实现的所有接口 `Class` 对象数组。

```
Class[] interfaces = clazz.getInterfaces();
```

7. 获取类的成员变量

通过 `Class` 对象的 `getFields()`方法获取到的是此 `Class` 对象所对应的实体的所有 `public` 字段(成员变量)。如果要获取所有的字段,可以使用 `getDeclaredFields()`方法。

```
Field[] fields = clazz.getDeclaredFields();
```

返回的是 `java.lang.reflect.Field` 类的对象数组。`Field` 类用来代表字段的详细信息。通过调用 `Field` 类提供的相应方法就可以获取字段的修饰符、数据类型、字段名等信息。

```
for (Field field : fields) { //循环处理每个字段
    String modifier = Modifier.toString(field.getModifiers()); //访问修饰符
    Class type = field.getType(); //数据类型
}
```



```
String name = field.getName(); //字段名
if(type.isArray()){ //如果是数组类型要特别处理一下
    String arrType = type.getComponentType().getName() + "[]";
    System.out.println("    " + modifier + " " + arrType + " " + name + ";");
}else{
    System.out.println("    " + modifier + " " + type + " " + name + ";");
}
}
```

8. 获取类的构造方法

通过 Class 对象的 `getConstructors()` 方法获取到的是该 Class 对象所对应的实体的所有 public 的构造方法。如果要获取所有的构造方法，可以使用 `getDeclaredConstructors()` 方法。

```
Constructor[] constructors = clazz.getDeclaredConstructors();
```

返回的是 `java.lang.reflect.Constructor` 类的对象数组。Constructor 类用来代表类的构造方法的相关信息。通过调用 Constructor 类提供的相应方法也可以获得该构造方法的修饰符、构造方法名、参数列表等信息。

```
for (Constructor constructor : constructors) {
    String name = constructor.getName(); //得到构造方法名
    String modifier = Modifier.toString(constructor.getModifiers());
    //得到访问修饰符
    System.out.print("    " + modifier + " " + name + "(");
    Class[] paramTypes = constructor.getParameterTypes(); //得到方法的参数列表
    for (int i = 0; i < paramTypes.length; i++) {
        if(i > 0){
            System.out.print(", ");
        }
        if(paramTypes[i].isArray()){ //处理参数类型为数组时的情况
            System.out.println(paramTypes[i].getComponentType().getName() + "[]");
        }else{
            System.out.print(paramTypes[i].getName());
        }
    }
    System.out.println(");");
}
```

9. 获取类的成员方法

通过 Class 对象的 `getMethods()` 方法获取到的是该 Class 对象所对应的实体的所有 public 成员方法。如果要获取所有的成员方法，可以使用 `getDeclaredMethods()` 方法。

```
Method[] methods = clazz.getDeclaredMethods();
```

返回的是 `java.lang.reflect.Method` 类的对象数组。Method 类用来代表类的成员方法的相关信息。通过调用 Method 类提供的相应方法也可以获得该成员方法的修改符、返回值类型、方法名、参数列表等信息。

```
for (Method method : methods) { //循环处理每个方法
```




```

String modifier = Modifier.toString(method.getModifiers()); //访问修饰符
Class returnType = method.getReturnType(); //返回类型
if(returnType.isArray()){ //如果是数组类型要特别处理一下
    String arrType = returnType.getComponentType().getName() + "[]";
    System.out.print("    " + modifier + " " + arrType + " " +
method.getName() + "(");
}else{
    System.out.print("    " + modifier + " " + returnType.getName() + " "
+ method.getName() + "(");
}
Class[] paramTypes = method.getParameterTypes(); //得到方法的参数 Class 数组
for (int i = 0; i < paramTypes.length; i++) {
    if(i > 0){
        System.out.print(", ");
    }
    if(paramTypes[i].isArray()){ //如果是数组类型要特别处理一下
        System.out.print(paramTypes[i].getComponentType().getName()+"[]");
    }else{
        System.out.print(paramTypes[i].getName());
    }
}
System.out.println(");");
}
}

```

以上代码的完整示例源代码可参阅随书光盘：Java 15\Src\com\qiujiy\corejava15\ReflectionTest.java。

15.2.2 创建对象

在前面所学的 Java 程序中，创建对象的方法通常都是通过 new 操作符调用该类的构造方法来创建的。如：

```
java.util.Date currentDate = new java.util.Date();
```

大多数情况下，这种方式已足够满足需求。因为在编译期间，已经知道要创建的对象所对应的类名称。但是，如果现在编写一个开发工具软件，将可能直到运行时才知道要创建的对象所对应的类名称。比如，一个 GUI 开发工具可以让用户拖取各种开发组件到设计界面上。这种情况下，需要完成的功能伪代码如下所示。

```

public Object create(String className){ //以类名作为参数传入
    根据类名来创建出它的对象
    返回这个新创建的对象
}

```

这时，可以通过 Java 反射 API，根据需要选择对应类的适当构造方法在运行时动态创建出对象。我们分两种情况来讨论利用反射创建对象的方式。

1. 使用无参构造方法

如果要使用无参数的构造方法创建对象，只需要调用这个类对应的 Class 对象的 newInstance()方法。



```
Class c = Class.forName("java.util.ArrayList");
List list = (List) c.newInstance();
```

需要注意的是：如果指定名称的类没有无参构造方法，在调用 `newInstance()` 方法时会抛出一个 `NoSuchMethodException` 异常。

【示例 1】如下是使用反射机制调用无参构造方法创建指定名称类的对象的示例。

```
package com.qiujy.corejava15;

import java.util.Date;

/** 使用反射机制调用无参构造方法创建指定名称类的对象 */
public class NoArgsCreateInstanceTest {
    public static void main(String[] args) {
        Date currentDate = (Date)newInstance("java.util.Date");
        System.out.println(currentDate);
    }

    public static Object newInstance(String className){
        Object obj = null;
        try {
            //加载指定名称的类，获取对应的 Class 对象，然后调用无参构造方法创建出一个对象
            obj = Class.forName(className).newInstance();
        } catch (InstantiationException e) {
            e.printStackTrace();
        } catch (IllegalAccessException e) {
            e.printStackTrace();
        } catch (ClassNotFoundException e) {
            e.printStackTrace();
        }
        return obj;
    }
}
```

2. 使用带参构造方法

要使用带参的构造方法来创建对象，首先需要获取指定名称的类对应的 `Class` 对象，然后通过反射获取满足指定参数类型要求的构造方法信息类(`java.lang.reflect.Constructor`)对象，调用它的 `newInstance` 方法来创建出对象。详细地说，可以分为如下 3 个步骤来完成。

第 1 步 获取指定类对应的 `Class` 对象。

第 2 步 通过 `Class` 对象获取满足指定参数类型要求的构造方法类对象。

第 3 步 调用指定 `Constructor` 对象的 `newInstance` 方法，传入对应的参数值，创建出对象。

我们知道 `java.util.Date` 类有一个带参数的构造方法，它需要一个 `long` 类型的参数。

【示例 2】通过反射使用 `java.util.Date` 类的带参构造方法来创建出一个对象。

```
package com.qiujy.corejava15;

import java.lang.reflect.Constructor;
import java.lang.reflect.InvocationTargetException;
import java.util.Date;
```




```

/** 利用反射使用指定带参构造方法创建指定名称类的对象 */
public class ArgsCreateInstanceTest {
    @SuppressWarnings("unchecked")
    public static void main(String[] args) {
        try {
            //第1步：加载指定名称的类，获取对应的 Class 对象
            Class clazz = Class.forName("java.util.Date");
            //第2步：获取具有指定参数类型的构造方法
            Constructor constructor = clazz.getConstructor(long.class);
            //第3步：给指定的构造方法传入参数值，创建出一个对象
            Date date = (Date)constructor.newInstance(123456789000L);
            System.out.println(date);
        } catch (ClassNotFoundException e) {
            e.printStackTrace();
        } catch (SecurityException e) {
            e.printStackTrace();
        } catch (NoSuchMethodException e) {
            e.printStackTrace();
        } catch (IllegalArgumentException e) {
            e.printStackTrace();
        } catch (InstantiationException e) {
            e.printStackTrace();
        } catch (IllegalAccessException e) {
            e.printStackTrace();
        } catch (InvocationTargetException e) {
            e.printStackTrace();
        }
    }
}

```

15.2.3 调用方法

使用反射可以取得指定类的指定方法的对象代表，方法的对象代表就是前面介绍的 `java.lang.reflect.Method` 类的实例，通过 `Method` 类的 `invoke` 方法可以动态调用这个方法。

`Method` 类的 `invoke` 方法的完整签名如下所示。

```

public Object invoke(Object obj, Object... args)
    throws IllegalAccessException, IllegalArgumentException,
    InvocationTargetException

```

这个方法的第一个参数是一个对象类型，表示要在指定的这个对象上调用这个方法；第二个参数是一个可变参数，用来给这个方法传递参数值；`invoke` 方法的返回值用来表示动态调用指定方法后的实际返回值。

提示

若要通过反射调用类的某个私有方法，可以在这个私有方法对应的 `Method` 对象上，先调用 `setAccessible(true)` 来取消 Java 语言对本方法的访问检查，然后再调用 `invoke` 方法来真正执行这个私有方法。



【示例 3】下面是一个通过反射来动态调用指定的方法的示例。

```
package com.qiujiy.corejava15;

import java.lang.reflect.InvocationTargetException;
import java.lang.reflect.Method;

/**利用反射来动态调用指定类的指定方法 */
@SuppressWarnings("unchecked")
public class ReflectInvokeMethodTest {
    public static void main(String[] args) {
        try {
            Class clazz = Class.forName("com.qiujiy.corejava15.Product");
            //利用无参构造方法创建一个 Product 的对象
            Product prod = (Product) clazz.newInstance();

            //获取名为 setName, 带一个类型为 String 的成员方法所对应的对象代表
            Method method1 = clazz.getDeclaredMethod("setName", String.class);
            //在 prod 对象上调用 setName, 并传值给它, 返回值是空
            Object returnValue = method1.invoke(prod, "爪哇");
            System.out.println("返回值: " + returnValue);
            //获取名为 displayInfo, 不带参数的成员方法所对应的对象代表
            Method method2 = clazz.getDeclaredMethod("displayInfo");
            method2.setAccessible(true); //取消访问检查
            //在 prod 对象上调用私有的 displayInfo 方法
            method2.invoke(prod);
        } catch (ClassNotFoundException e) {
            e.printStackTrace();
        } catch (SecurityException e) {
            e.printStackTrace();
        } catch (NoSuchMethodException e) {
            e.printStackTrace();
        } catch (IllegalArgumentException e) {
            e.printStackTrace();
        } catch (IllegalAccessException e) {
            e.printStackTrace();
        } catch (InvocationTargetException e) {
            e.printStackTrace();
        } catch (InstantiationException e) {
            e.printStackTrace();
        }
    }
}

class Product {
    private static long count = 0;
    private long id;
    private String name = "无名氏";
    public Product(){
        System.out.println("默认的构造方法");
        id = ++count;
    }
}
```



```

public long getId() {      return id; }
public void setId(long id) {      this.id = id; }
public String getName() {      return name; }
public void setName(String name) {
    System.out.println("调用 setName 方法");
    this.name = name;
}
private void displayInfo(){ //私有方法
    System.out.println(getClass().getName() + "[id=" + id + ",name=" + name + "]);
}
}

```

运行后，在控制台的输出结果为：

```

默认构造方法
调用 setName 方法
返回值: null
com.qiujuy.corejava15.Product[id=1,name=爪哇]

```

15.2.4 访问成员变量的值

使用反射可以取得类的成员变量的对象代表，成员变量的对象代表是 `java.lang.reflect.Field` 类的实例，可以使用它的 `getXXX` 方法来获取指定对象上的值，也可以调用它的 `setXXX` 方法来动态修改指定对象上的值，其中的 `XXX` 表示成员变量的数据类型。

【示例 4】通过反射来动态设置和获取指定对象指定成员变量的值。

```

package com.qiujuy.corejava15;

import java.lang.reflect.Field;

/**利用反射来动态获取或设置指定对象的指定成员变量的值 */
public class ReflectFieldTest {
    @SuppressWarnings("unchecked")
    public static void main(String[] args) {
        try {
            Class c = Class.forName("com.qiujuy.corejava15.Product");
            //使用无参构造方法创建对象
            Product prod = (Product)c.newInstance();
            //调用私有属性
            Field idField = c.getDeclaredField("id");
            idField.setAccessible(true); //取消对本字段的访问检查
            //设置 prod 对象的"idField"成员变量的值为 100
            idField.setLong(prod, 100);
            //获取 prod 对象的"idField"成员变量的值
            System.out.println("id=" + idField.getLong(prod));
            Field nameField = c.getDeclaredField("name");
            nameField.setAccessible(true);
            nameField.set(prod, "张三");
            System.out.println("name=" + nameField.get(prod));
        } catch (ClassNotFoundException e) {

```



```

        e.printStackTrace();
    } catch (InstantiationException e) {
        e.printStackTrace();
    } catch (IllegalAccessException e) {
        e.printStackTrace();
    } catch (SecurityException e) {
        e.printStackTrace();
    } catch (NoSuchFieldException e) {
        e.printStackTrace();
    }
}
}

```

15.2.5 操作数组

数组也是一个对象，可以通过反射来查看数组的各个属性信息。

【示例 5】通过反射来获取数组信息。

```

package com.qiujiy.corejava15;

/** 反射获取数组信息 */
public class ReflectArrayTest {
    public static void main(String[] args) {
        short[] sArr = new short[5]; //创建数组
        int[] iArr = new int[5];
        long[] lArr = new long[5];
        float[] fArr = new float[5];
        double[] dArr = new double[5];
        byte[] bArr = new byte[5];
        boolean[] zArr = new boolean[5];
        String[] strArr = new String[5];
        System.out.println("short 数组类: " + sArr.getClass().getName());
        //直接获取数组的类型名
        System.out.println("int 数组类: " + iArr.getClass().getName());
        System.out.println("long 数组类: " + lArr.getClass().getName());
        System.out.println("float 数组类: " + fArr.getClass().getName());
        System.out.println("double 数组类: " + dArr.getClass().getName());
        System.out.println("byte 数组类: " + bArr.getClass().getName());
        System.out.println("boolean 数组类: " + zArr.getClass().getName());
        System.out.println("String 数组类: " + strArr.getClass().getName());
    }
}

```

运行此程序，在控制台的输出结果为：

```

short 数组类: [S
int 数组类: [I
long 数组类: [J
float 数组类: [F
double 数组类: [D

```




```
byte 数组类: [B
boolean 数组类: [Z
String 数组类: [Ljava.lang.String;
```

直接获取数组对应的 Class 对象的全限定名时, 返回的是 “[x]” 的形式。要想真正获取数组的类型名, 可以使用 `getComponentType()` 方法获取此数组类型的 Class 对象, 然后再调用 `getName()` 方法来获取全限定名:

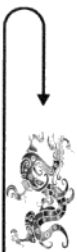
```
package com.qiujy.corejava15;

/** 反射获取数组信息 */
public class ReflectArrayTest {
    public static void main(String[] args) {
        short[] sArr = new short[5]; //创建数组
        int[] iArr = new int[5];
        long[] lArr = new long[5];
        float[] fArr = new float[5];
        double[] dArr = new double[5];
        byte[] bArr = new byte[5];
        boolean[] zArr = new boolean[5];
        String[] strArr = new String[5];
        //通过 getComponentType() 方法获取此数组类型的 Class, 再获取它的全限定名
        System.out.println("short 数组类: " +
sArr.getClass().getComponentType().getName());
        System.out.println("int 数组类: " +
iArr.getClass().getComponentType().getName());
        System.out.println("long 数组类: " +
lArr.getClass().getComponentType().getName());
        System.out.println("float 数组类: " +
fArr.getClass().getComponentType().getName());
        System.out.println("double 数组类: " +
dArr.getClass().getComponentType().getName());
        System.out.println("byte 数组类: " +
bArr.getClass().getComponentType().getName());
        System.out.println("boolean 数组类: " +
zArr.getClass().getComponentType().getName());
        System.out.println("String 数组类: " +
strArr.getClass().getComponentType().getName());
    }
}
```

这次运行的结果为:

```
short 数组类: short
int 数组类: int
long 数组类: long
float 数组类: float
double 数组类: double
byte 数组类: byte
boolean 数组类: boolean
String 数组类: java.lang.String
```

数组也可以使用反射动态创建, 主要是利用 `java.lang.reflect.Array` 类来操作的。



【示例 6】利用反射动态创建数组。

```
package com.qiujiy.corejava15;

import java.lang.reflect.Array;

/** 利用反射动态创建数组的示例 */
public class ReflectCreateArrayTest {
    public static void main(String[] args) {
        Object obj = Array.newInstance(int.class, 5);
        //动态创建一个长度为5的 int 类型数组
        for(int i = 0; i < 5; i++){ //动态设置数组元素的值
            Array.setInt(obj, i, i * 10);
        }
        for (int i = 0; i < 5; i++) { //动态获取数组元素的值
            System.out.println("第"+i+"号元素的值: "+Array.getInt(obj, i));
        }
    }
}
```

15.3 反射与动态代理

代理模式是 Java 中很常用的一种设计模式，在企业应用高级框架中大量用到。它的原理主要就是应用到反射。接下来就来详细介绍这种模式。

15.3.1 静态代理

在某些情况下，一个客户不想或者不能直接引用另一个对象，需要通过代理对象来间接操作目标对象，代理就在客户端和目标对象之间起到中介的作用。

举一个示例来说明这个问题：有一个客户想找一个厂家做一批衣服，但客户找不到合适的厂家，于是通过一个中介公司，由中介公司帮他找厂家做这些衣服，当然中介公司要从中收取一定的中介费，我们用 Java 程序来模拟完成这个问题。

首先，定义一个能完成生产一批衣服功能的接口。

```
package com.qiujiy.corejava15.proxy;

/** 服装厂接口 */
public interface ClothingFactory {
    /** 有"生产一批衣服"的功能 */
    void productClothing();
}
```

“LiNingCompany”公司是一家能真正生产这一批服装的公司。

```
package com.qiujiy.corejava15.proxy;

/** LiNing 公司就是一家能生产服装的公司 */
```




```
public class LiNingCompany implements ClothingFactory {
    public void productClothing() {
        System.out.println("生产出一批 LiNing 服装");
    }
}
```

“ProxCompany”是一家专门帮人介绍服装公司的中介公司,它需要收取一定的中介费。

```
package com.qiujuy.corejava15.proxy;

/** 专门为别人找服装厂的中介公司 */
public class ProxyCompany implements ClothingFactory {
    private ClothingFactory cf;
    //中介公司自己不会生产服装,需要找一家真正能做服务的公司
    public ProxyCompany(ClothingFactory cf){
        this.cf = cf;
    }
    public void productClothing() {
        System.out.println("收取 10000 元的中介费");
        cf.productClothing(); //委托真正的服务公司生产服装
    }
}
```

最后,客户通过中介公司生产了这一批服装。

```
package com.qiujuy.corejava15.proxy;

/** 客户 */
public class Customer {
    public static void main(String[] args) {
        //通过中介公司生产一批服装
        ClothingFactory cf = new ProxyCompany(new LiNingCompany());
        cf.productClothing();
    }
}
```

运行这个程序,在控制台得到如下输出结果。

```
收取 10000 元的中介费
生产出一批 LiNing 服装
```

仔细分析这个应用程序,可以把代理模式用图 15.1 来表示。

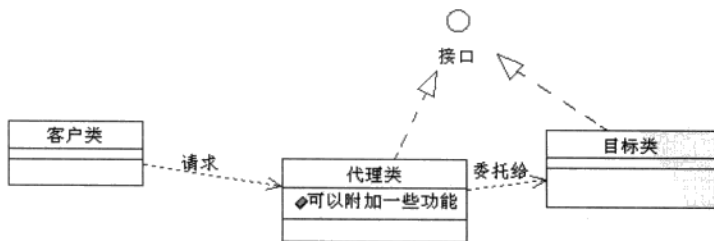


图 15.1 代理模式原理图



上面程序的做法，使用的代理模式是静态代理模式，它的特征是代理类和目标对象的类都是在编译期间就已经确定下来的，不利于程序的扩展。上面示例中，如果客户还想找一个“生产一批鞋”的工厂，那么还需新增一个代理类和一个目标类。如果客户还需要很多其他的服务，就必须一一新增代理类和目标类。有没有办法来解决这个问题呢？答案是使用动态代理。

15.3.2 动态代理

动态代理的原理就是，在程序运行时根据需要动态创建目标类的代理对象。JavaSE API 在 `java.lang.reflect` 包中提供了对动态代理的支持的类和接口，主要包括如下。

(1) `InvocationHandler` 接口：代理类的处理类都需要实现这个接口。接口中只有一个方法。

```
public Object invoke(Object proxy, Method method, Object[] args) throws
Throwable;
```

在实际使用时，第一个参数 `proxy` 指代理类；第二个参数 `method` 是被代理的方法的 `Class` 对象；第三个参数 `args` 为传给该方法的参数值数组。这个抽象方法在代理类中动态实现。

(2) `Proxy` 类：提供用于创建动态代理类和实例的静态方法。

【示例 7】利用 JDK 对动态代理的支持来创建一个动态代理的处理类。

```
package com.qiujy.corejava15.proxy;

import java.lang.reflect.*;

/** 动态代理处理类 */
public class DynaProxyHandler implements InvocationHandler {
    /** 目标对象 */
    private Object target;
    /** 创建一个目标对象的代理对象 */
    public Object newProxyInstance(Object target) {
        this.target = target;
        /*
        第一个参数：定义代理类的类加载器
        第二个参数：代理类要实现的接口列表
        第三个参数：指派方法调用的调用处理程序
        */
        return Proxy.newProxyInstance(this.target.getClass().getClassLoader(),
            this.target.getClass().getInterfaces(), this);
    }
    public Object invoke(Object proxy, Method method, Object[] args) throws
    Throwable {
        Object result = null;
        try {
            // 目标对象上的方法调用之前可以添加其他代码.....
            result = method.invoke(this.target, args);
            // 通过反射来调用目标对象上对应的方法
            // 目标对象上的方法调用之后可以添加其他代码.....
        }
    }
}
```



```

    } catch (Exception e) {
        throw e;
    }
    return result; // 把方法的返回值返回给调用者
}
}

```

在客户端的调用代码改为如下方式。

```

public class Customer {
    public static void main(String[] args) {
        // 动态代理方式
        DynaProxyHandler handler = new DynaProxyHandler();
        ClothingFactory cf2 = (ClothingFactory) handler.newProxyInstance(new
        LiNingCompany());
        cf2.productClothing();
    }
}

```

这个动态代理处理类 DynaProxyHandler 没有和具体的目标类耦合，可能适用于任何的目标类，从而提高了代理的扩展性。

总之，动态代理的原理就是利用反射机制在运行时动态创建目标对象的代理对象。对于初学者来说，理解动态代理是比较困难的，建议读者对照 JavaSE API 帮助文档多阅读几遍“DynaProxyHandler”的代码，看懂即可。

15.4 本章练习

(1) 利用反射机制编写一个程序，这个程序能把类中所有的成员变量赋值，并把操作信息输出到控制台。

(2) 利用反射机制编写一个程序，这个程序能指定调用类的某个方法，并把操作信息输出到控制台。

(3) 利用反射机制编写一个程序，这个程序能指定调用类的构造方法，并把操作信息输出到控制台。

(4) 某同学在面试时遇到这么一个题“请编写一段代码，提供获取某个对象属性的方法”，他编写的代码如下所示，请补充完整。

```

Public Object get Property(Object owner, String fieldname) throws Excoption {
    Class oclass = Oowner.getClass();
    Field field = _____
    Object property = _____
    return property;
}

```



